

# An Algorithm-Hardware Co-design Based on Revised Microscaling Format Quantization for Accelerating Large Language Models

Yingbo Hao<sup>†\*</sup>, Huangxu Chen<sup>‡\*</sup>, Yi Zou<sup>†§</sup>, Yanfeng Yang<sup>†</sup>

<sup>†</sup> South China University of Technology, <sup>‡</sup> Hong Kong University of Science and Technology (GZ), Guangzhou, China

<sup>†</sup>{202410193423@mail., zouyi@, mi\_yyf@mail.}scut.edu.cn, <sup>‡</sup>{hchen499@connect.}hkust-gz.edu.cn

**Abstract**—The narrow-bit-width data format is crucial for reducing the computation and storage costs of modern deep learning applications, particularly in large language models (LLMs) based applications. Microscaling (MX) format has been proven as a drop-in replacement for the baseline FP32 in existing inference frameworks, with low user friction. However, deploying such a new format into existing hardware systems is still challenging, and the dominant solution for LLM inference at low precision is still low-bit quantization. This particularly limits the strategic applications of such LLMs in real deployment on a large scale. In this work, we propose an algorithm-hardware co-design that adopts a two-level *Revised MX Format Quantization* (RMFQ) and a *Revised MX Format Accelerator* (RMFA) architecture design. RMFQ proposes the *revised MX (RMX)* format and provides a novel quantization framework with innovative group direction. Also, RMFA provides an RMX adaptive hardware architecture and an RMX encoding scheme. As a result, RMFQ pushes the limit of 4-bit and 6-bit quantization to a new state-of-the-art, and RMFA surpasses the existing outlier-aware accelerator such as OliVe, achieving a  $1.28\times$  speedup and a  $1.31\times$  energy reduction.

**Index Terms**—Large Language Model (LLM), Revised Microscaling (RMX) Format, Accelerator, Two-Level Quantization.

## I. INTRODUCTION

The latest advances in Artificial Intelligence (AI) capabilities, such as conversational question answering and intelligent code completion [1], are primarily achieved by scaling up the underlying deep learning models [2]. However, this scaling has led to a significant increase in the demand for more computational power and memory capacity to keep up with training and deploying such large-scale models.

Meanwhile, model quantization has been proven to be an effective technique for reducing memory usage and computational requirements by representing values with narrower bit widths [3]. Native support for low bit width formats is now prevalent in AI-focused hardware. For example, NVIDIA has introduced INT8 integer type in tensor core since its Turing architecture [4]. Particularly, 8-bit quantization necessitates a scale factor for each tensor and has demonstrated noteworthy impact when the number of model parameters is relatively small [5], [6]. However, when dealing with Transformer-based *Large Language Models* (LLMs), the model's performance is easily affected by a very small portion ( $< 0.1\%$ ) of outliers [7], [8]. For example, regular quantization methods fail at a scale of 6.7B parameters for OPT model [8].

\*These authors have contributed equally to this work.

§Corresponding author.

TABLE I  
QUANTIZATION ARCHITECTURE COMPARISON

| Method             | Encoding Scheme     | Off / On-chip Memory<br>Aligned | Bit Width | Compute<br>Bit Width | Overhead Area Ratio |
|--------------------|---------------------|---------------------------------|-----------|----------------------|---------------------|
| OLAccel [12]       | Coordinate list     | No                              | 5.81      | 4.36                 | 71%                 |
| GOBO [11]          | Coordinate list     | No                              | 4.04/6.81 | 16                   | 55%                 |
| OliVe [10]         | Outlier-victim pair | Yes                             | 4.23/8    | 4.23/8               | 4%                  |
| <b>RMFA (Ours)</b> | RMX detecting       | Yes                             | 4.06/6.06 | 4.06/6.06            | 2%                  |

Additionally, tensor-level scaling has been shown to be insufficient for Transformer-based LLMs due to their limited dynamic range. Various methods have been proposed to address outliers, with one common approach being to handle normal values and outliers differently [9]–[11]. For example, OliVe [10] adopts an outlier-victim pair (OVP) quantization to sacrifice normal values to accommodate outliers locally. GOBO [11] and OLAccel [12] use coordinate list to distinguish outliers from normal values. However, such progress not only leads to unaligned memory storage and accesses but also causes large area overheads due to the extra outlier controller.

Another approach is to use new data formats to simultaneously accommodate outliers and regular values. Research has shown that using micro-scaled data formats, which associate scaling factors with fine-grained sub-blocks of a tensor, is a promising method to quantize Transformer-based LLMs [13]–[16], and even more effective in the sub-8 bit regime [14]–[16]. However, these efforts are all optimized for existing GPUs, which are architecturally not compatible with these new data formats, often requiring complex dynamic quantization algorithms such as data recognition and reordering. Microscaling (MX) data format [17] has been proposed as the first open standard for a family of micro-scaled data format aimed at deep learning inference. However, the deployment of MX still has a long way to go through the combined efforts of industries and academia.

In this paper, we propose an algorithm-hardware co-design that adopts a two-level *Revised MX Format Quantization* (RMFQ) and a *Revised MX Format Accelerator* (RMFA) architecture design to solve the low-precision quantization problem of LLMs. In the software algorithm, we design a novel quantization framework based on *revised MX (RMX)* format to handle outliers more effectively. At the same time, in the hardware architecture, we design RMFA to support the computation of RMX format, mainly including a systolic

array adapted to RMX format and an RMX encoding scheme. Table I compares RMFQ with previous outlier-aware quantization architecture. RMFA achieves high performance with minimal hardware overhead and can be efficiently integrated into existing hardware accelerators. The main contributions are listed as follows:

- We **propose** RMFQ, a two-level quantization method using RMX format that introduces adaptive int/float and a novel group direction. The method significantly reduces the equivalent bit width compared to the original MX format and handles outliers in LLMs more effectively.
- We **design** RMFA to benefit from RMFQ, mainly including an RMX adaptive systolic array architecture and a RMX encoding scheme. This architecture optimizes data transfer and storage, thus effectively reducing the bandwidth requirements for data transmission.
- We **implement** and **demonstrate** the effectiveness of RMFQ on representative LLMs. On one hand, RMFQ achieves state-of-the-art (SOTA) quantization accuracy for LLMs of 4-bit and 6-bit. On the other hand, evaluation of RMFA showcases  $3.13\times$ ,  $1.88\times$ , and  $1.28\times$  speedup over the ANT, OLAccel and OliVe, respectively.

## II. BACKGROUND AND PROPOSED METHOD

In this section, we summarize the challenges in quantizing LLMs and propose the RMX format. Based on this, we raise our novel quantization method.

### A. Challenges in Quantizing LLMs

There have been various quantization methods proposed over the past years. The two main classes of methods are post-training quantization (PTQ) [3], [7], [10], [13], [15] and quantization-aware training (QAT) [3], [9], [18]. QAT requires fine-tuning and enables lower bit quantization [19]. However, LLMs retraining is extremely resource-consuming, which makes PTQ a more desirable method for LLMs. Recent state-of-the-art PTQ methods [7], [15] demonstrate that the weights in LLMs can be quantized to four or even three bits via grouping-based (e.g., a scaling factor for  $g$  consecutive weights in a tensor) quantization methods with competitive model performance.

Nevertheless, quantizing activations with the same method doesn't work well due to the existence of outliers. A mainstream approach is to use higher precision for calculations involving outliers [8], [14], [16], but this simultaneously brings complex dynamic quantization perception processes, thus reducing the speed and making system implementation difficult. Fig. 1 visualizes the absolute values of activation and weight tensors of a typical linear layer in OPT-6.7B [20]. It can be seen that outliers only exist in specific channels of activation. The intuitive approach is to use per-channel (e.g., a scaling factor for each channel in an activation tensor) quantization for activations. However, per-channel activation quantization does not map well to hardware-accelerated GEMM kernels. Assume a typical per-channel quantization using INT4. The quantization process can be expressed as:

$$X^{INT4} = \text{Round}\left(\frac{X^{FP16}}{\Delta}\right), \Delta = \frac{\max(|X^{FP16}|)}{2^{4-1} - 1}, \quad (1)$$

where  $x^{FP16}$  is the tensor to be quantized,  $x^{INT4}$  is the quantized counterpart and  $\Delta$  is the quantization step size. The matrix multiplication process can be expressed as:

$$Y = \text{diag}(\Delta_x^{FP16}) \cdot (X^{INT4} \cdot W^{INT4}) \cdot \text{diag}(\Delta_w^{FP16}). \quad (2)$$

In those kernels, scaling can only be performed along the outer dimensions of the matrix multiplication (i.e., token dimension of activations). Therefore, previous work all uses per-token or per-tensor activation quantization, even if there is work using a per-channel smoothing factor to keep mathematical equivalence of a linear layer, which is still complicated [13].

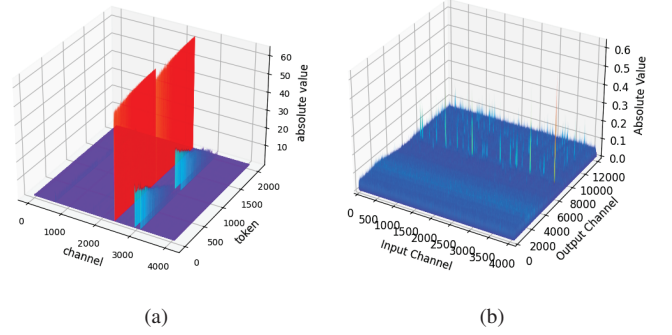


Fig. 1. Absolute values of (a) activation and (b) weight tensors in OPT-6.7B.

### B. RMX Format

MX formats have demonstrated their effectiveness on inference of various deep learning benchmarks, including LLMs [21]. However, the approach relies on pseudo-simulation, and current GPU architectures are not compatible with MX format ultimately. Therefore, integrating MX format into existing quantization methods and proposing practical hardware-adaptive architectures is crucial to fully unlocking the potential of MX format. Based on this analysis, we propose the RMX format. Fig. 2 compares MX format with our proposed RMX format.

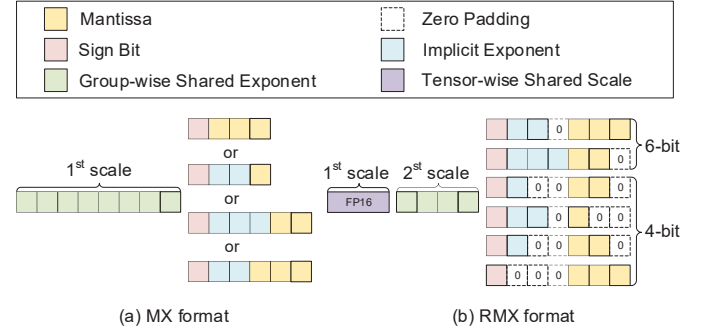


Fig. 2. Comparison between MX and RMX format

1) *Mixed Types and Precisions*: Firstly, we replace the fixed integer (e.g., MXINT4) or floating-point (e.g., MXFP4, MXFP6) number with a mixed int/float type and 4-bit/6-bit precision format. This is based on the observation that in different layers of a neural network, the distribution characteristics of each tensor are different, possibly being uniform-like, Gaussian-like, or Laplace-like [9]. It is not appropriate to use the same data type for different layers, such as using MXINT4

for activation tensors. Moreover, the modification is hardware-friendly. All these 6 data types can be implemented in a unit supporting E3M3 (3 implicit exponent, 3 mantissa) for *multiplication and accumulation* (MAC) operations. Note that zero padding is used to illustrate the computing mechanism of RMX format; 4/6-bit RMX format remains its original bit width in memory storage and access.

2) *Reduced Equivalent Bit Width  $bit_e$* : Secondly, we replace the one-level group-wise scale factor with the two-level tensor-wise plus group-wise scale factor. Since both MX and RMX formats are micro-scaled formats that have small granularities, the group-wise scale factors have a considerable memory overhead. To this end, we propose a new metric denoted as  $bit_e$  to calculate the equivalent bit width of micro-scaled formats and measure the memory-saving effects.  $bit_e$  is defined as:

$$bit_e = \frac{bit \cdot g + bit_s}{g}, \quad (3)$$

where  $bit$  is the quantization bit width,  $g$  is the group size and  $bit_s$  is the bit width of the group-wise scale factor. Assume  $g = 64$ , which we will illustrate in Section IV-B.

For example, in 4-bit quantization, assume an activation tensor of [4096, 2048] for OPT-6.7B on WikiText [22]. The average bit width burden of the tensor-wise scale factor on each group is  $16 \div \frac{4096 \cdot 2048}{64} = 10^{-4}$ , which is too small to be considered in  $bit_e$ . Therefore,  $bit_s$  is crucial to  $bit_e$ . As shown in Fig. 3, we calculate the max shared exponent and shared exponent overflow ( $> 2^3 - 1 = 7$ ) for OPT-6.7B and GPT2-XL [1]. It is evident that the shared exponent of the weights rarely experiences overflow. However, there is a significant amount of overflow in the activations. Therefore, we choose 4 as our group-wise shared exponent bit width for a max shared exponent of 15. As a result,  $bit_e$  for the original MX format is  $\frac{4 \cdot 32 + 8}{64} = 4.25$ , and  $bit_e$  for the proposed RMX format is  $\frac{4 \cdot 64 + 4}{64} = 4.06$ .

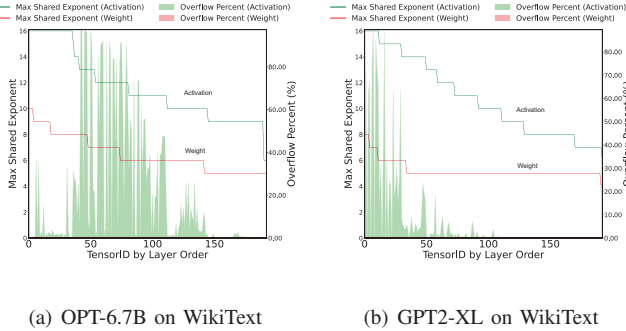


Fig. 3. Max shared exponent and shared exponent overflow percent for representative LLMs. The overflow percent for weights is too small, thus hard to see in the figure.

### C. Quantization Method

Our intuition is to leverage the implicit scale factor in the RMX format, enabling group-wise quantization to be performed along the inter dimensions of matrix multiplication. Fig. 4 illustrates the proposed two-level quantization method. We use the typical per-tensor method for 1<sup>st</sup> quantization. To handle the channel-wise outliers of activation tensors,

we revise the previous group direction of activation tensors; instead, we group along the channel direction. For weight tensors, we find the same pattern of distribution compared to activation tensors, which is shown in Fig. 1. Therefore, we group along the output channel direction to handle the less significant outliers in weight tensors too. Nonetheless, we propose the hardware system that is compatible with such group direction in Section III. Therefore, the 1<sup>st</sup> 4-bit quantization process can be expressed as:

$$\bar{X}^{FP16} = \frac{X^{FP16}}{\Delta^{1st}}, \Delta^{1st} = \frac{\max(|x^{FP16}|)}{\max(|Q^{grid}|)}, \quad (4)$$

where  $Q^{grid}$  is the quantization grid (e.g., [-8, ..., 6, 7] for RMXINT4),  $\bar{X}^{FP16}$  is the 1<sup>st</sup> quantization result. The 2<sup>nd</sup> quantization process for a group can be expressed as:

$$\frac{1}{\Delta^{2nd}} = \max(\lfloor \log_2 \frac{\max(|Q^{grid}|)}{\max(|\bar{X}_{group}^{FP16}|)} \rfloor - 1, 15), \quad (5)$$

$$X^{RMX4} = Round(\frac{\bar{X}_{group}^{FP16}}{\Delta^{2nd}}), \quad (6)$$

where  $\bar{X}_{group}^{FP16}$  is the 1<sup>st</sup> quantization result of a group,  $\lfloor \cdot \rfloor$  is the rounding-to-floor function and  $X^{RMX4}$  is the 2<sup>nd</sup> quantization result. We can obtain the optimal group-wise shared exponent by performing the above quantization process.

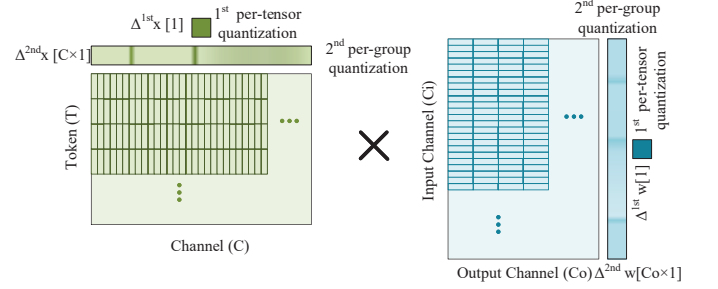


Fig. 4. Illustration of the proposed two-level revised MX format quantization, i.e., RMFQ, method.

We also adopt *Mean Square Error* (MSE) as the quantization optimization target, which has been shown to be effective by many previous studies [3], [9], [23]. The MSE metric can be defined by the following equation:

$$MSE = \int (X^{FP16} - X^{RMX4})^2 p(x) dx,$$

where  $p(x)$  is the probability density function. By searching the minimum MSE, we can select the optimal RMX data types for each layer and quantize layers with the greatest MSEs with higher precision, that is, do mixed data types/precisions quantization.

### III. HARDWARE ARCHITECTURE

In this section, we describe in detail the contributions from this work in the aspects of RMFA, mainly including an improved systolic array and a hardware-friendly RMX encoding scheme.



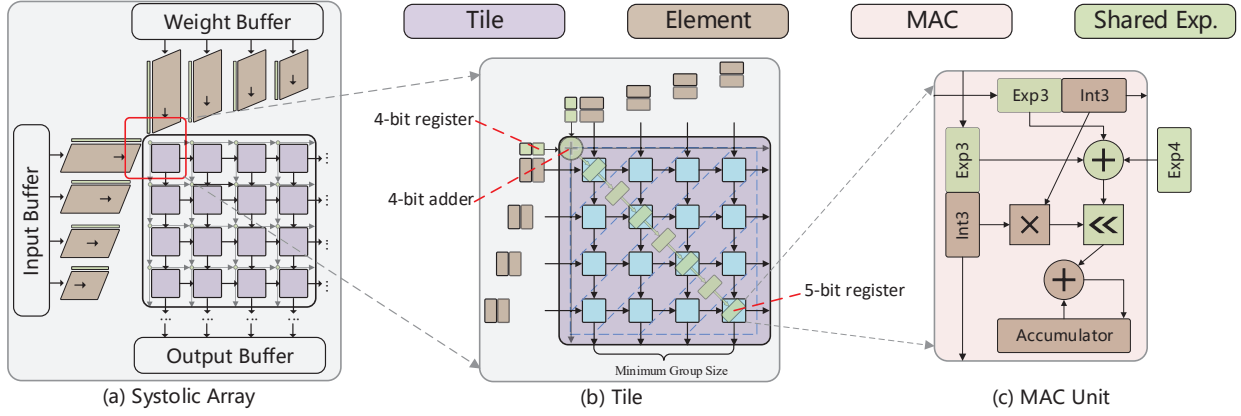


Fig. 5. A brief overview of the systolic array architecture for RMFA.

#### A. Systolic Array Architecture for RMFA

Fig. 5(a) presents the proposed systolic array architecture. Given that the group size may vary from the array size, it becomes necessary to segment the systolic array into tiles of the smallest viable group size. Each tile computes one group independently per cycle, with the shared exponent being propagated in a systolic fashion. It is evident that a reduction in group size leads to an increase in the number of tiles, thereby incurring more hardware overhead for a given array size. For instance, with an array size of  $a = 16$  and a group size of  $g = 4$ , the additional hardware requirements include  $(\frac{a}{g})^2 \times (2g - 1) = 112$  5-bit registers,  $2(\frac{a}{g})^2 = 32$  4-bit registers, and  $(\frac{a}{g})^2 = 16$  4-bit adders. Nonetheless, while decreasing the group size may enhance precision, it also significantly increases both hardware and bandwidth overheads.

Fig. 5(b) depicts a tile designed for the RMX format with an example group size of  $g = 4$ . Due to the intrinsic characteristics of the systolic array, the next columns of data must be transmitted to the next column of MAC units one cycle later. Consequently, data belonging to the same group is initially partitioned through a grid, with corresponding shared exponents denoted by identical colors. The matrix operation formula for the MX data type is expressed as:

$$\sum_{1}^n \sum_{1}^g 2^{e_s^a} a_{mx} \times 2^{e_s^w} w_{mx} = \sum_{1}^n \sum_{1}^g (a_{mx} \times w_{mx}) \ll e_s^a + e_s^w,$$

where  $\ll$  means bit shifting operation,  $e_s^a$  and  $e_s^w$  are the shared exponents for activation and weight respectively,  $a_{mx}$  and  $w_{mx}$  are activations and weights in either RMX4 or RMX6,  $n$  and  $g$  are group number and group size. Therefore, we first sum the shared exponents of the activations and weights, followed by shifting the resultant product.

The modified MAC unit is illustrated in Fig. 5(c). In contrast to the conventional MAC unit, the proposed model necessitates the substitution of the dual-input shifter with a triple-input shifter and an expansion in the accumulator's width to accommodate the shift range pressure induced by the shared exponents.

#### B. Hardware-friendly RMX Encoding Scheme

The computation of the shared exponent is pivotal in the RMFA. Once determined, the encoding and decoding algo-

rithms can be straightforwardly applied based on the RMX format specification. The quantization circuit is depicted in Fig. 6. Data resulting from nonlinear operations, typically in FP16 or BF16 format, is initially scaled by a predetermined factor from the software calibration phase and truncated to a 16-bit integer. This truncation preserves precision for groups with smaller magnitudes. For each data group, the circuit identifies the position of the first '1' within the top 16 bits, encoding this position as a 4-bit integer to ascertain the group's shared exponent. Subsequently, the data is right-shifted, truncated to 8 bits, and quantized to RMX data according to the specified RMX type.

#### Algorithm 1 RMX Encoding Algorithm

**Input:** Group\_Data<sub>G,N</sub>, Scale, RMX\_Type

**Output:** Group\_Data<sub>G,N</sub>, Shared\_Exponent<sub>N</sub>

- 1: Group\_Data<sub>G,N</sub>  $\leftarrow$  Group\_Data<sub>G,N</sub> \* Scale
- 2: Group\_Max\_Data<sub>N</sub>  $\leftarrow$  Max(Group\_Data<sub>G,N</sub>, dim = 0)
- 3: Max\_Data  $\leftarrow$  Max(Group\_Max\_Data<sub>N</sub>)
- 4: Shared\_Exponent<sub>N</sub>  $\leftarrow$  Frexp( $\frac{Max\_Data}{Group\_Max\_Data\_N}$ )
- 5: Group\_Data<sub>G,N</sub>  $\leftarrow$  Group\_Data<sub>G,N</sub> \* ( $2^{Shared\_Exponent\_N}$ )
- 6: Group\_Data<sub>G,N</sub>  $\leftarrow$  Quantize(Group\_Data<sub>G,N</sub>, RMX\_Type)
- 7: **return** Group\_Data<sub>G,N</sub>, Shared\_Exponent<sub>N</sub>

At the software level, the primary goal is to simulate the error introduced by the quantization algorithm. The algorithm 1 outlines the RMX encoding process. Initially, group data undergoes scaling. Subsequently, the sharing index is determined by comparing the maximum value within the group to the global maximum value, utilizing the *Frexp* function to extract the exponent of the floating-point number. Finally, all data are quantized to integers, with shifts applied based on the shared index, to approximate the error induced by tensor quantization.

During hardware inference, the quantization circuit is depicted in Fig. 6. Data resulting from nonlinear operations, typically in FP16 or BF16 format, is initially scaled by a predetermined factor from the software calibration phase and truncated to a 16-bit integer. This truncation preserves precision for groups with smaller magnitudes. For each data group, the circuit identifies the position of the first '1' within the top 16 bits, encoding this position as a 4-bit integer to ascertain the group's shared exponent. Subsequently, the data

is right-shifted, truncated to 8 bits, and quantized to RMX data according to the specified *RMX\_type*.

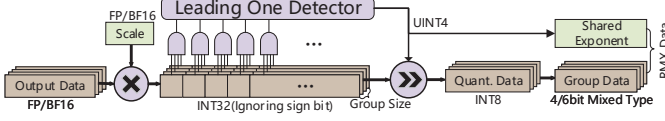


Fig. 6. The Hardware Architecture of the RMX Encoder

#### IV. EXPERIMENT

##### A. Methodology

**Quantization Framework and Benchmark** To evaluate our proposed RMFQ framework, we deploy our experiments on PyTorch. To demonstrate the general applicability of our proposed method on LLMs, we evaluate our method on the OPT suite [20], LLaMA [24], Llama-2 [25] with varying model sizes ranging from 6.7B to 13B, and GPT2-XL [1]. Outliers become a significant problem when model size scales up to about 6.7B. We evaluate language modeling tasks using the WikiText (Wiki) [22] and C4 [26] datasets. We use perplexity as an evaluation metric, which is a widely used one for autoregressive models; lower perplexity means better model performance. Additionally, to prove the generality of our method, we conduct evaluations on ResNet-18 [27] using the ImageNet dataset [28]; higher accuracy means better model performance.

**Quantization Baselines** We compare our framework with existing quantization frameworks, including ANT [9], OliVe [10], and Tender [29]. ANT is a quantization framework that introduces a novel data type to better represent data and adapt to different data formats between layers. OliVe is a hardware-friendly quantization framework that uses the OVP encoding method to protect outlier values while ensuring memory alignment. Tender is a decomposed quantization technique that uses powers of two to represent scale factors of decomposed matrices to avoid explicit requantization. We conduct PTQ experiments for evaluation. Each the PTQ quantization uses 128 randomly sampled calibration datasets.

**Accelerator Baselines** We compare the performance and energy of the RMFQ accelerator with three hardware-friendly quantization accelerators, including OliVe, ANT, and OLAccel. We use 4-bit and 6-bit mixed precision quantization to ensure the same perplexity as the other three quantization frameworks.

**RMFQ Implementation** Our evaluation of the proposed systolic array and encoder is conducted using the Synopsys Design Compiler (DC) tool [30]. We opt for the TSMC N28 technology node, setting the timing constraint to a relaxed  $2ns$  to facilitate a fair comparison in terms of area and power consumption metrics. Furthermore, we standardize both the systolic array and group sizes to 64 for consistency in our analysis. Additionally, we use CACTI [31] to evaluate the area and power consumption of the buffer. We perform performance and energy simulations using a cycle-level simulator, DnnWeaver [32]. We use DeepScaleTool [33] to scale the other designs to the same technology node for a fair comparison.

##### B. Quantization Performance Results

**Group Size Exploration** We utilize both CNN models and Transformer-based LLMs for computer vision and natural language processing tasks. Experimental results for 4-bit RMX on WikiText are shown in Fig. 7. It is evident that the group size ranging from 8 to 64 has negligible impact on models' accuracy, and the effect on 6-bit RMX can only be less significant. However, as the group size continues to increase, the impact on the model's accuracy becomes significant.

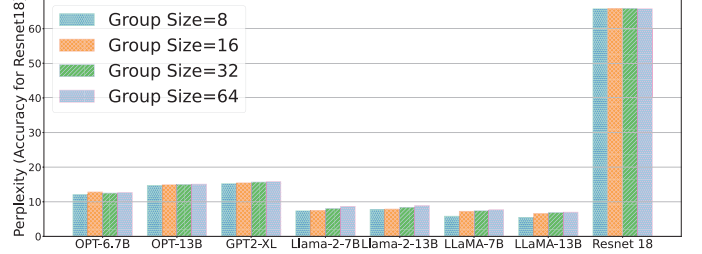


Fig. 7. Accuracy Comparison of Different Group Size

To further demonstrate the superiority of RMFQ, we conduct experiments on the data types and group directions using the OPT-6.7B model on WikiText. The experiment result is shown in Fig. 8. We only show the quantization method for activation because the quantization method for weight is paired. It can be seen that token-wise group quantization that we adopt demonstrates greater robustness with increasing group size, and the 4-bit mixed RMX format outperforms RMXINT4. Additionally, channel-wise group quantization also achieves competitive results with small group sizes, indicating the potential of RMFQ integration in existing GEMM kernels (e.g., tensor cores).

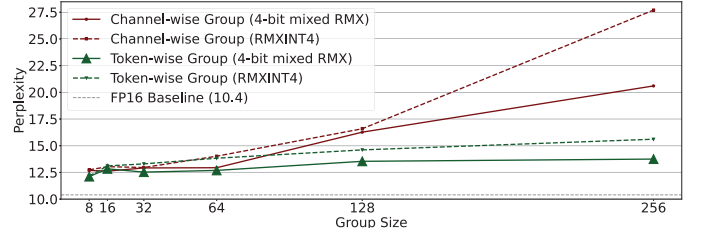


Fig. 8. Accuracy Comparison under Different Circumstances

**Benchmark and Comparison** For 4-bit RMFQ, we benchmark against INT8, while for 6-bit RMFQ, we benchmark against FP16. The results are presented in Table II. To summarize, the proposed RMFQ framework sets a new SOTA for 4-bit and 6-bit quantization of LLMs. In all LLM tasks, 4-bit RMFQ achieves performance surpassing INT8, while 6-bit RMFQ is almost lossless (maximum perplexity increase of 3.4% compared to the FP16 baseline). Compared to 6-bit quantization, the impact of outliers on model performance is significantly greater in 4-bit quantization. This is due to the fact that quantizing outliers with others results in a larger scale factor. This effect becomes more pronounced at the inherently lower quantization levels of 4-bit PTQ. Our proposed RMFQ allows outliers and normal values to be quantized in isolated groups and provides smaller quantization granularities. Across all scenarios, our approach surpasses existing works, including ANT, OliVe, and Tender. 4-bit RMFQ shows far better

TABLE II  
PERFORMANCE METRICS OF VARIOUS MODELS AND CONFIGURATIONS

| Method             | OPT-6.7B     |              | OPT-13B      |              | GPT2-XL      |              | Llama-2-7B  |             | Llama-2-13B |             | LLaMA-7B    |             | LLaMA-13B   |             | Resnet-18    |              |
|--------------------|--------------|--------------|--------------|--------------|--------------|--------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|--------------|--------------|
|                    | Wiki ↓       | C4 ↓         | Wiki         | C4           | Wiki         | C4           | Wiki        | C4          | Wiki        | C4          | Wiki        | C4          | Wiki        | C4          | PTQ ↑        | QAT ↑        |
| INT8               | 16.91        | 17.28        | 4E+3         | 4E+3         | 15.51        | 17.35        | 8.79        | 9.96        | 20.52       | 22.23       | 8.52        | 9.73        | 7.49        | 8.61        | 69.38        | 69.64        |
| 4-bit ANT          | 4E+4         | 4E+4         | 4E+3         | 5E+3         | 25.39        | 29.57        | 142.73      | 161.34      | 113.27      | 119.32      | 60.56       | 66.71       | 73.75       | 80.32       | 59.52        | 65.67        |
| 4-bit OliVe        | 43.44        | 32.41        | 45.61        | 37.56        | 16.36        | 18.14        | 34.24       | 38.85       | 36.73       | 40.12       | 28.64       | 30.83       | 24.12       | 25.77       | 59.49        | 65.61        |
| 4-bit Tender       | 13.56        | 13.71        | 16.43        | 17.33        | 16.12        | 17.93        | 36.47       | 41.72       | 55.08       | 28.14       | 23.85       | 25.26       | 13.68       | 14.11       | 64.23        | 66.41        |
| <b>4-bit RMFQ*</b> | <b>12.69</b> | <b>12.30</b> | <b>15.11</b> | <b>15.96</b> | <b>15.83</b> | <b>17.48</b> | <b>7.39</b> | <b>8.25</b> | <b>7.83</b> | <b>8.97</b> | <b>6.95</b> | <b>7.91</b> | <b>6.56</b> | <b>7.62</b> | <b>65.74</b> | <b>67.79</b> |
| FP16               | 10.40        | 10.63        | 10.13        | 10.32        | 14.69        | 16.30        | 5.83        | 6.36        | 5.25        | 6.43        | 5.86        | 6.97        | 5.19        | 5.49        | 69.68        | 69.87        |
| 6-bit ANT          | 11.76        | 11.90        | 11.42        | 11.31        | 14.84        | 16.46        | 6.23        | 6.91        | 6.03        | 7.29        | 6.82        | 7.93        | 6.71        | 6.93        | 68.99        | 69.54        |
| 8-bit OliVe        | 10.93        | 10.73        | 10.28        | 10.47        | <b>14.70</b> | 16.37        | 6.16        | 6.62        | 5.98        | 7.01        | 6.79        | 7.52        | 6.23        | 7.03        | 68.82        | 69.45        |
| 8-bit Tender       | 10.93        | 10.76        | <b>10.17</b> | <b>10.39</b> | 14.72        | 16.35        | 6.09        | 6.53        | 5.43        | 6.61        | 6.05        | 7.24        | 5.41        | 5.58        | 69.21        | 69.89        |
| <b>6-bit RMFQ</b>  | <b>10.48</b> | <b>10.67</b> | <b>10.21</b> | <b>10.38</b> | 14.72        | <b>16.33</b> | <b>6.03</b> | <b>6.47</b> | <b>5.39</b> | <b>6.57</b> | <b>6.03</b> | <b>7.12</b> | <b>5.34</b> | <b>5.52</b> | <b>69.30</b> | <b>70.02</b> |

\* Presenting best results using a group size of 8 for best 4-bit quantization method exploration, which is close to results with a group size of 64. Note that RMFQ still outperforms all other methods with a group size of 64.

perplexity than others, especially in Llama-2 and LLaMA. 6-bit RMFQ results are comparable to those of 8-bit OliVe and Tender and surpass them in most situations.

### C. Accelerator Implementation Results

**Area and Power** Table III presents the architectural configuration of MXFA. At an operating frequency of 500 MHz, MXFA occupies an area of  $3.09 \text{ mm}^2$  and consumes  $472.5 \text{ mW}$  of power. The systolic array integrates the RMX format decoding circuitry as described by Section II-B. To ensure sufficient computational throughput, we design a 64 KB input buffer, a 32 KB weight buffer, and a 32 KB output buffer, with the input buffer having double the capacity to facilitate ping-pong operations. Additionally, we standardize the number of MAC units and buffer sizes across all baseline designs. For a fair comparison, we set the accumulator bit width of all MAC unit designs to 24 bits for subsequent performance and energy evaluations.

**Performance** Fig. 9(a) shows the normalized cycles of different accelerator designs across various models. ANT exhibits the worst performance due to the lack of protection for outliers. Although OLAccel employs mixed precision to protect outliers, its performance is hindered by unaligned memory access and bandwidth limitations. OliVe uses OVP encoding to protect outliers with negligible bandwidth overhead, but this sacrifices accuracy, leading to poor performance when trying to restore the same accuracy. RMFQ achieves better accuracy with negligible bandwidth overhead by sharing exponents, resulting in superior performance at the same accuracy level. Overall, in LLMs, RMFA achieves performance improvements of  $3.13\times$ ,  $1.88\times$ , and  $1.28\times$  compared to ANT, OLAccel and OliVe, respectively.

**Energy** Fig. 9(b) shows the static and dynamic energy consumption of different accelerator designs across various models, including DRAM, on-chip buffers, and cores. Due to the hardware-friendly systolic array design of RMFA, it exhibits the lowest energy consumption. In comparison, ANT and OLAccel consume the most energy due to mixed precision, which resulted in more memory accesses and computation cycles. Compared to OliVe, the sacrifice of accuracy leads to more high-precision layers and higher energy consumption. Overall, in LLMs, RMFA achieves energy reductions of  $2.38\times$ ,  $1.86\times$ , and  $1.31\times$  compared to ANT, OLAccel and OliVe, respectively.

TABLE III  
AREA AND POWER OF RMFA

| Component       | Setup      | Area/ $\text{mm}^2$ | Power/ $\text{mW}$ |
|-----------------|------------|---------------------|--------------------|
| Systolic Array* | 64x64 MACs | 1.41                | 249.9              |
| Encoder         | 64         | 0.03                | 3.0                |
| Input Buffer    | 64KB       | 0.49                | 49.5               |
| Weight Buffer   | 32KB       | 0.65                | 94.6               |
| Output Buffer   | 32KB       | 0.51                | 75.5               |
| <b>Total</b>    |            | <b>3.09</b>         | <b>472.5</b>       |

\* Decoding circuits of RMX format is included in the systolic array.

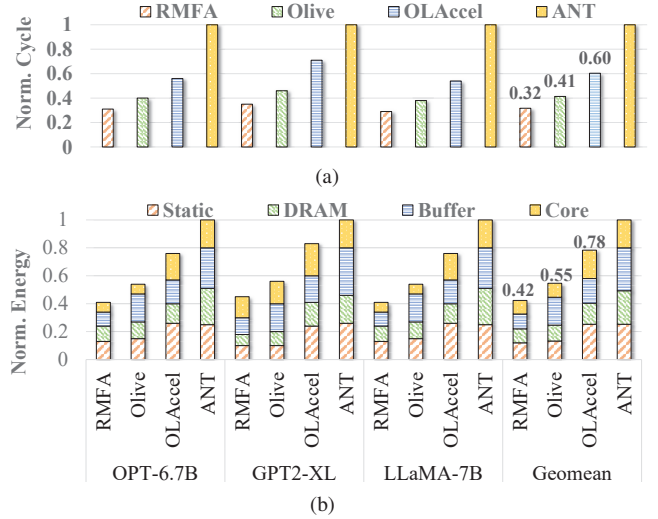


Fig. 9. Comparison of the normalized (a) cycle and (b) energy different accelerator designs

## V. CONCLUSIONS

In this paper, we propose an algorithm-hardware co-design for accelerating LLMs. To reduce the quantized equivalent bit width and allow for mixed data types/precision quantization, we propose the RMX format. To handle outliers in LLMs more effectively, we propose the two-level RMFQ with novel group direction. Furthermore, we design RMFA to efficiently support the RMX format. In conclusion, by integrating the RMFQ for low-bit quantization like 4/6-bit, RMFA realizes significantly  $1.28\times$  performance and  $1.31\times$  energy efficiency improvement in LLMs compared to the prior design such as OliVe with negligible accuracy loss.

## VI. ACKNOWLEDGMENT

This research is supported partly by SBSTR Fund No. KJZD2024090310270801, as well as partly by the SCUT Fund No. K3200890.



## REFERENCES

- [1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, “Language models are unsupervised multitask learners,” *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [3] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. Van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” *arXiv preprint arXiv:2106.08295*, 2021.
- [4] J. Burgess, “Rtx on—the nvidia turing gpu,” *IEEE Micro*, vol. 40, no. 2, pp. 36–44, 2020.
- [5] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, “I-bert: Integer-only bert quantization,” in *International conference on machine learning*. PMLR, 2021, pp. 5506–5518.
- [6] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, “Q-bert: Hessian based ultra low precision quantization of bert,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 8815–8821.
- [7] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for on-device llm compression and acceleration,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [8] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 318–30 332, 2022.
- [9] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1414–1433.
- [10] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [11] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, “Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.
- [12] E. Park, D. Kim, and S. Yoo, “Energy-efficient neural network accelerator based on outlier-aware low-precision computation,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 688–698.
- [13] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
- [14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [15] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, “Qserve: W4a8kv4 quantization and system co-design for efficient llm serving,” *arXiv preprint arXiv:2405.04532*, 2024.
- [16] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, “Atom: Low-bit quantization for efficient and accurate llm serving,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.
- [17] B. Darvish Rouhani, R. Zhao, V. Elango, R. Shafipour, M. Hall, M. Mesmakhosroshahi, A. More, L. Melnick, M. Golub, G. Varatkar *et al.*, “With shared microexponents, a little shifting goes a long way,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.
- [18] O. Zafri, G. Boudoukh, P. Izsak, and M. Wasserblat, “Q8bert: Quantized 8bit bert,” in *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*. IEEE, 2019, pp. 36–39.
- [19] K. Chowdhary and K. Chowdhary, “Natural language processing,” *Fundamentals of artificial intelligence*, pp. 603–649, 2020.
- [20] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [21] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf *et al.*, “Microscaling data formats for deep learning,” *arXiv preprint arXiv:2310.10537*, 2023.
- [22] W. contributors, “68-95-99.7 rule - wikipedia,” in *The Free Encyclopedia*. [Online], 2022.
- [23] R. Banner, Y. Nahshan, and D. Soudry, “Post training 4-bit quantization of convolutional networks for rapid-deployment,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [24] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [25] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [26] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.
- [27] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [28] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [29] J. Lee, W. Lee, and J. Sim, “Tender: Accelerating large language models via tensor decomposition and runtime requantization,” *arXiv preprint arXiv:2406.12930*, 2024.
- [30] P. Kurup and T. Abbasi, *Logic Synthesis Using Synopsys*, 2nd ed. Springer Publishing Company, Incorporated, 2011.
- [31] N. Muralimanohar, R. Balasubramonian, and N. Jouppi, “Cacti 6.0: A tool to model large caches. hp laboratories,” *Tech. Rep. HPL-2009-85*, 2009.
- [32] H. Sharma, J. Park, D. Mahajan, E. Amaro, J. K. Kim, C. Shao, A. Mishra, and H. Esmaeilzadeh, “From high-level deep neural models to fpgas,” in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Ieee, 2016, pp. 1–12.
- [33] S. Sarangi and B. Baas, “Deepscaletool: A tool for the accurate estimation of technology scaling in the deep-submicron era,” in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. Ieee, 2021, pp. 1–5.